Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 11

Disk Management Utilities

Lab 11 guide (bonus): measure capacity, length and allocation, and build and inspect a disk image safely.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Read filesystem capacity and per-file allocation, without mixing up df, du and apparent size.
2. Create a bounded sparse regular image, and explain the difference between length and allocation.
3. Format and inspect only an image you own, or read a supplied metadata trace and state its limits.

THIS LAB

Lab 11, bonus

The whole lab is optional bonus work. Skipping it carries no penalty. First the concepts, then the lab. Mounting and resizing are extensions, so they come last.

DISCUSSION

A file is 8 MB long. Does it use 8 MB of the disk?

What if most of it was never written? Which command would tell you, and which would only tell you the length?

CONCEPTS FIRST

Three sizes of a file

Three ideas: the capacity of a filesystem, the length of a file, and the space really allocated to it.

Capacity, length and allocation

Size	Tool	Answers
Capacity and free space	`df -hT`	How big is the whole filesystem, and how much is free?
Apparent length	`ls -l, stat %s`	How many bytes does the file claim to have?
Allocated space	`du -h, stat %b x %B`	How many disk blocks does the file really occupy?

The three can differ. df covers a whole mounted filesystem, not one file. A sparse file can be long and small at the same time.

A sparse file has holes

student@server: ~/oslab-work/lab11

```
$ truncate -s 8M images/demo-sparse.img
$ dd if=/dev/zero of=images/demo-written.img bs=1M count=8 status=none
$ ls -lh images/demo-*.img
$ du -h images/demo-*.img
```

truncate -s 8M sets the length to 8 MiB without writing the bytes. The unwritten part is a hole: reads give zeros, and no block is used.

dd copies bytes. Here it writes 8 MiB of real zeros, so every block is allocated.

Both files have the same length. du shows the difference.

Reading stat

student@server: ~/oslab-work/lab11

```
$ stat -c '%n length=%s blocks=%b block-unit=%B' images/demo-*.img
images/demo-sparse.img length=8388608 blocks=0 block-unit=512
images/demo-written.img length=8388608 blocks=16384 block-unit=512
```

%s is the length in bytes. %b is the number of blocks, and %B is the size of one counted block.

Allocated bytes = %b times %B. Here: 16384 times 512 is 8 MiB for the written file, and nothing for the sparse one.

The exact figures depend on the host filesystem, and compression or deduplication can change them.

A disk image is a regular file

IMAGE

A file that acts as a disk

A disk image is an ordinary file whose bytes are laid out like a disk. A tool can format it like a disk.

SAFETY

Check before you format

Make sure it is a regular file, not a link, and that it is inside your own workspace. Never point a tool at a /dev/ device.

FORMAT

Formatting adds metadata

mkfs writes structures that describe the filesystem. The length stays the same, but some blocks are now allocated.

LAB 11 (BONUS)

Measure before you resize

QuantumTech's build server is short of storage. Rehearse safe measurements on your own image files.

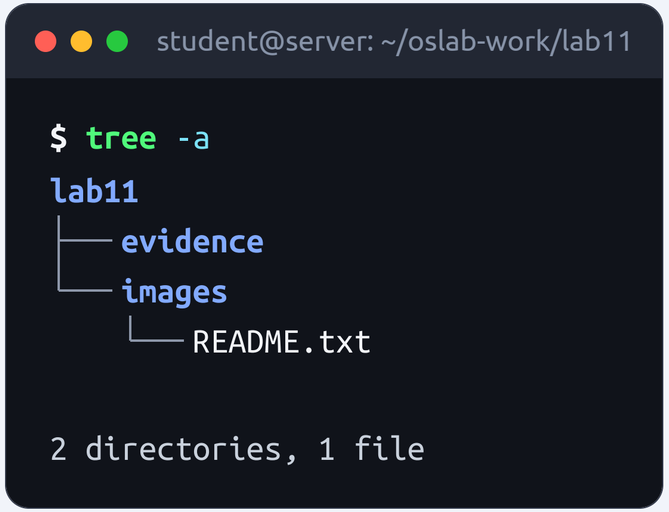
The scenario

SCENARIO

QuantumTech's build server

The server is running low on storage. Rehearse safe capacity measurements on regular image files you own, and tell apparent length from allocated blocks before anyone thinks about mounting or resizing.

This whole lab is bonus work. Skipping it carries no penalty.



Inside the 120 minutes

Minutes	What happens
0 to 10	Introduction, objectives and setup
10 to 25	Levels 0 and 1, guided: inventory and usage
25 to 35	Written prediction
35 to 50	Level 2: create and inspect an image

Minutes	What happens
50 to 70	Level 3: format and inspect metadata
70 to 85	Tests and feedback
85 to 100	Individual changed-case checkpoint
100 to 120	Correction, cleanup and submission

How you work

Your own account

Log in with your own account. Use only your own files and processes. Never use a /dev/ device, a partition or a mount point. Only regular files in your workspace.

AI tools

Optional in the investigation and the tests. Not allowed in the prediction and the individual checkpoint.

Evidence

Keep what you observe in the evidence folder. Short, attributable evidence earns more than long output.

Initial prediction accuracy is lightly weighted. A reasoned attempt and an evidence-based correction earn credit.

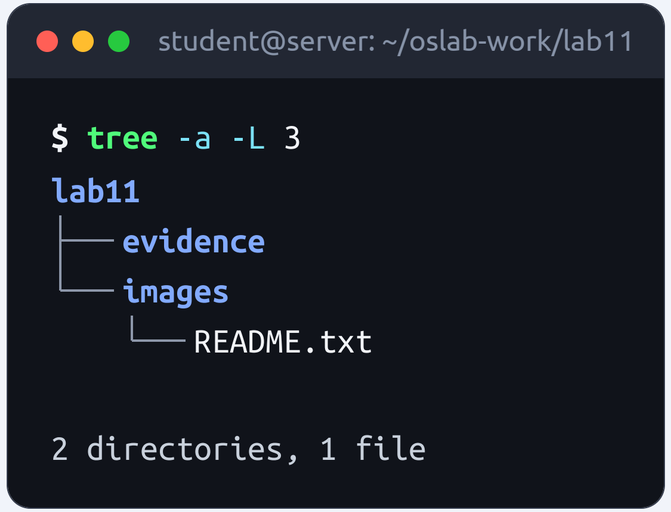
Setup: start the lab

student@server: ~

```
$ whoami
$ command -v oslab
$ oslab doctor
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab11
$ cd "$OSLAB_WORKSPACE/lab11"
$ pwd
$ mkdir -p evidence
$ find . -maxdepth 3 -type f
```

- whoami shows your account. command -v oslab and oslab doctor check the helper.
- If the helper is missing, follow the local setup guide in the course repository.
- oslab start lab11 makes the lab folder. It keeps your work if you start again, and it does not change your current folder, so cd into it yourself.
- If you are resuming, look at the existing files before you run a command that writes to them.

Compare with the starting tree



images/README.txt is a reminder about regular image files.

The images you make go in images/. They are large, so they are not submission files.

The helper also writes a hidden file, .oslab-managed.json. Leave it as it is.

GUIDED

Inventory and usage

Levels 0 and 1, with the instructor: fifteen minutes.

Step 1: capacity, read only

student@server: ~/oslab-work/lab11

```
$ df -hT "$PWD"
$ lsblk -o NAME,SIZE,TYPE,MOUNTPOINT
```

df reports the filesystem that holds the current folder: its size, its type and how much is free.

lsblk lists the devices you can see. In WSL or a container the view can differ.

CAREFUL

Only look

These are read-only observations. They are not permission to format any device in the list.

Step 2: two files of equal length

student@server: ~/oslab-work/lab11

```
$ truncate -s 8M images/demo-sparse.img
$ dd if=/dev/zero of=images/demo-written.img bs=1M count=8 status=none
$ ls -lh images/demo-*.img
$ du -h images/demo-*.img
$ stat -c '%n length=%s blocks=%b block-unit=%B' images/demo-*.img
```

/dev/zero is only a source of zero bytes. Both output paths are regular files you own.

Both lengths are 8 MiB. The sparse file normally uses fewer blocks.

Step 3: compute the allocation

TRY

Compute

Allocated bytes = %b times %B. Compare the result with the length %s for each file.

THINK

Observe

Does df show the allocation of one file? Does du have to equal ls -l for a sparse file?

PREDICT

Guess first, then try

Before you run the experiment. Ten minutes.

The prediction

PREDICTION

Write this down first

A 32 MiB file is created with truncate without writing all its bytes. Predict its apparent size, and whether its allocated space must also be 32 MiB. Explain what the holes represent.

HOW

The rules

Alone, and without AI. On paper or in the course worksheet, before you run the command. You may compare with a neighbour for five minutes, but keep your own answer.

A wrong prediction costs nothing. It is marked for being made and for the evidence-based correction afterwards.

LEVEL 2

Create and inspect an image

Fifteen minutes, from minute 35 to 50.

Lab 11

Step 1: a bounded image, and proof it is regular

student@server: ~/oslab-work/lab11

```
$ truncate -s 32M images/scratch.img
$ test -f images/scratch.img && test ! -L images/scratch.img
$ readlink -f images/scratch.img
$ stat -c '%F %n length=%s blocks=%b block-unit=%B' images/scratch.img
$ du -h images/scratch.img
```

test -f is true for a regular file. test ! -L is true when it is not a symbolic link. readlink -f prints the real path.

Confirm all three before any command that formats. Never use a /dev/ path, a partition or a mount point instead.

Step 2: save the sizes

student@server: ~/oslab-work/lab11

```
$ {
$   stat -c '%n length=%s blocks=%b block-unit=%B' images/*.img
$   du -h images/*.img
$   df -hT "$PWD"
$ } > evidence/sizes.txt
```

Record the length and the allocation of every image, with the filesystem capacity.

THINK

Explain

Compare the new image with the two guided files. Why is the length of a file not the space reserved for it?

LEVEL 3

Format and inspect metadata

Twenty minutes, from minute 50 to 70.

Lab 11

Step 1 and 2: check the tools, then format only the image

student@server: ~/oslab-work/lab11

```
$ command -v mkfs.ext4
$ command -v dumpe2fs
$ if test -f images/scratch.img && test ! -L images/scratch.img; then
$     mkfs.ext4 -F images/scratch.img
$ fi
$ dumpe2fs -h images/scratch.img > evidence/filesystem.txt 2>&1
```

Use this only if both tools exist. mkfs.ext4 replaces the contents of the image, so do it once, on a new image you own.

-F lets mkfs work on a regular file. It is not a safety check. The fixed path and the test before it are the safety.

dumpe2fs -h prints the header of the filesystem. Then run stat and du again on the image and compare.

Step 3: read the header

1. Find the filesystem UUID, the block count, the free blocks and the block size.
2. Estimate the filesystem size: block count times block size.
3. Explain why not every block is free for your data: some hold metadata or are reserved.

DONE

Complete when

Your measurements tell capacity, length and allocation apart. The image stays bounded and regular. Your metadata claims match a real header or a supplied trace that you label as such.

Step 4: no tools? Use a labelled trace

student@server: ~/oslab-work/lab11

```
$ cat > evidence/filesystem.txt <<'TRACE'
$ Illustrative ext4 header excerpt; not measured on this machine.
$ Filesystem UUID: 11111111-2222-3333-4444-555555555555
$ Block count: 32768
$ Free blocks: 25830
$ Block size: 1024
$ TRACE
```

Save the excerpt as it is, with its label. In your report write: metadata interpretation only; formatting not executed.

The numbers describe a 32 MiB filesystem with some space used or reserved. They say nothing about your real image or free space.

Three comparisons

Fifteen minutes, from minute 70 to 85.

The three comparisons, and their limits

Case	Evidence	Claim and limit
Sparse and fully written, 8 MiB	Length and blocks of each	Allocation depends on the content and the host filesystem
Fresh and formatted, 32 MiB	Size before and after, and the header	Formatting adds metadata. It does not show that FUSE works
Capacity	df for the filesystem that holds the image	It covers the whole filesystem, not only the image

Add the allocation of the formatted image to evidence/sizes.txt. If you used a trace, label it as supplied evidence, not a local test. FUSE is not needed here, because this core never mounts.

If something does not work

A large image and reset

oslab reset refuses an archive over 2 MB. Save your evidence and remove your own unmounted image files first, or keep the workspace.

No tools

Without mkfs.ext4 or dumpe2fs, the sparse-file work and the checkpoint still work. Use the labelled trace.

Stay safe

Only regular files in your workspace. See man du and man mkfs.ext4.

INDIVIDUAL CHECKPOINT

Show what you can do alone

Fifteen minutes, from minute 85 to 100. No AI, no neighbours.

Why there is a checkpoint

It checks you, not the tools

Everything before it may use AI and help. The checkpoint shows what you understood without them.

A short question on paper

You answer one question on paper or on the course worksheet. Your earlier work does not need to be finished.

Practice question

The question on the next slide is public practice. A graded session may use a fresh private variant.

The question

CHECKPOINT

Practice question

Two files have the same apparent length. One was made sparsely with truncate, the other was fully written with zero bytes. Predict how their allocated blocks may differ, and name a command and field that checks it. Why is df not enough to say which file uses the space?

What to give

1. The result or diagnosis.
2. The mechanism.
3. One observation that could check it.

The instructor collects your answer before discussing the public key.

Look back, then hand in

Ten minutes of correction, ten to clean up and submit.

Correct your prediction

1. Keep your original prediction in view. Under it, write confirmed or corrected.
2. Cite the test that shows it.
3. Explain the OS mechanism in three to five sentences.

ANSWER

Three questions

(a) Why can a 32 MiB image occupy fewer allocated bytes? (b) Why does formatting change allocation? (c) What did you observe here, and what needs real FUSE or server validation?

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab11/evidence"
$ cd "$OSLAB_WORKSPACE/lab11"
$ cp -- evidence/sizes.txt evidence/filesystem.txt "$SUBMISSION_REPO/lab11/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

Keep the large image files in your workspace. They are not submission files. filesystem.txt is a real header or a trace you labelled.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab11
├── evidence
│   ├── filesystem.txt
│   └── sizes.txt
└── README.md

1 directory, 3 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

How it is marked: 10 points

Evidence for the lab objectives	Points
Bounded regular-image measurements and metadata interpretation, correct	3
Sparse and written, fresh and formatted comparisons, with the limits stated	2
df, du and length explained, and the filesystem metadata; original prediction and correction	2
Individual checkpoint: result or diagnosis, and mechanism	2
Concise, attributable evidence and the required artifacts	1

This lab is bonus work, so its points are extra. Other valid commands and names earn credit. If you use AI, note one useful suggestion and how you verified it.

Common mistakes

Mistake	What you see	Fix
A /dev/ path instead of your image	A real device could be damaged	Only a regular file in your workspace
Reading length as space used	A sparse file looks bigger than it is	Compare %s with %b times %B
Formatting a file you still need	Its contents are gone	Format only a new image, once
Calling a trace a measurement	A claim your evidence cannot back	Label supplied traces as such

Mounting, checking and growing

Optional extensions. They need FUSE or a prepared machine, and never run on the shared server's real devices.

Mount an image, then check and grow it

FUSE

Mount without root

A FUSE tool can show the files inside an image under a folder you own. Unmount it as soon as you have looked.

CHECK

Check

e2fsck -n reads the filesystem and reports problems without changing anything.

GROW

Grow

truncate makes the file longer, then resize2fs grows the filesystem to fit. Each step changes what df reports.

A small diagnostic utility

UTILITY

A script of your own

Print df, the largest folders with du, and a warning when usage passes a threshold you choose.

THRESHOLDS

Read thresholds with care

80 percent means different things on a 10 GB disk and a 10 TB disk. Think about what remains, and how fast it fills.

CLEANUP

Always clean up

Unmount, and remove your own image files, before you finish.

Key takeaways

01

Three sizes

Capacity is for a filesystem, length is for a file, and allocation is the space really used.

02

Sparse files

Holes read as zeros and use no blocks, so length and allocation can differ a lot.

03

Only your own regular files

Never aim a format command at a device. Check the path first.

04

Say what you measured

A header you read is evidence. A supplied trace is only an illustration.

Next session

COMING UP

The end of the labs

Look back at your reports, and bring the questions you could not answer yourself.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 14, File-System Implementation.